MODERN MONK LABS — PHYSICAL AI RESEARCH INTERNSHIP

PID Control: From Mathematical Principles to Robust Robotics Hardware Implementation

Author: Candidate: Devanshu Raut | **Target System:** Second-Order Mass-Spring-Damper | **Runtime:** Python 3.10+ (NumPy, Matplotlib)

Executive Summary & Project Scope

In Physical AI, autonomous machines cannot operate reliably on raw perception alone. The gap between digital algorithmic decisions and physical reality is bridged by closed-loop feedback control. This technical report presents a ground-up, production-grade realization of a Proportional-Integral-Derivative (PID) control architecture engineered without external control toolboxes. We model, simulate, and validate the system on an assigned second-order physical plant (m = 1.0 kg, c = 10.0 N·s/m, k = 20.0 N/m) under hard actuator constraints (±30 N). Furthermore, we expose, quantify, and resolve four fatal real-world hardware non-idealities: integral windup, sensor jitter chatter, derivative kick, and discretization latency. Finally, classical Ziegler-Nichols system identification is applied and critically evaluated.

Section 1: The PID Concept, Daily-Life Analogy & Architecture

The Intuitive Analogy (Automobile Highway Cruise Control): Imagine driving a car up a steep mountain highway with target speed 100 km/h.

- **Proportional Action (*P*):** Determines *how far off you are right now*. If the vehicle slows to 80 km/h (error = 20 km/h), the driver steps down firmly on the accelerator pedal. If error is small, the corrective push is small.
- **Integral Action (*I*):** Determines *how long you have been falling short*. As the vehicle climbs the steep grade, gravity pulls backward. A purely proportional throttle creates a stalemate where engine thrust balances gravity at 92 km/h. The integral term continuously sums this persistent error over time, gradually depressing the pedal until the car reaches exactly 100 km/h.
- **Derivative Action (*D*):** Determines *how rapidly you are closing the gap*. As the car rushes toward 100 km/h at high acceleration, the derivative senses the closing rate and eases off the throttle ahead of time, preventing sudden overshoot.

Closed-Loop Control Architecture with Hardware Non-Idealities

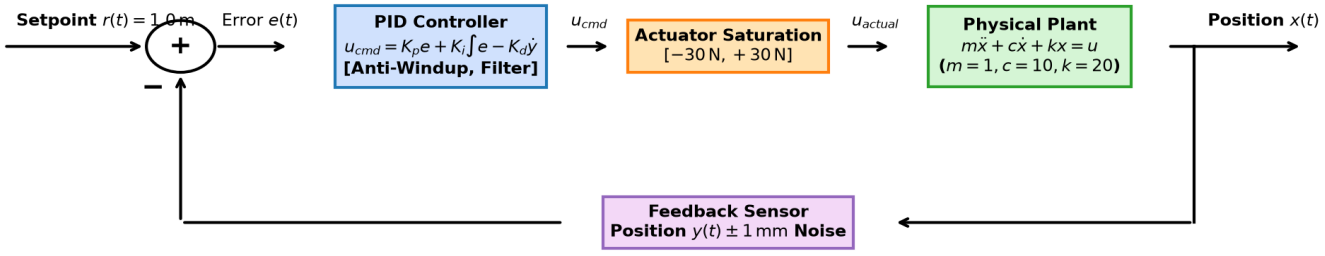


Figure 1: Full closed-loop feedback architecture showing reference setpoint, controller with protective filters, saturation bounds, continuous physical plant, and sensor feedback.

Section 2: Mathematical Formulation, Dimensional Units & Discrete Implementation

In continuous time, the standard textbook PID control law generates actuator command $u(t)$ from tracking error $e(t) = r(t) - y(t)$:

$$u(t) = K_p \cdot e(t) + K_i \cdot \int_0^t e(\tau)\, d\tau + K_d \cdot [de(t) / dt]$$

Physical Dimensional Unit Analysis (SI Consistency)

In physical robotics, control gains are not dimensionless scalars; they are physical transfer coefficients connecting displacement errors in metres [m] to output forces in Newtons [N]:

Symbol	Parameter Name	SI Units	Physical Interpretation in Mechanics
e(t)	Tracking Error	m (metres)	Instantaneous spatial deviation between target and cart position.
Kp	Proportional Gain	N / m	Acts as an artificial software spring pulling the cart toward target.
Ki	Integral Gain	N / (m·s)	Accumulates persistent historical error to eliminate static spring droop.
Kd	Derivative Gain	N·s / m	Acts as an artificial viscous damper (shock absorber) opposing velocity.
u(t)	Control Force	N (Newtons)	Total net force commanded to the electromechanical actuator.

Discrete-Time Digital Computer Implementation

Embedded microcontrollers cannot execute continuous calculus. Time is sliced into discrete ticks $t_k = k \cdot \Delta t$. Numerical integration and differentiation must be approximated:

1. **Proportional Term:** Direct algebraic evaluation: P[k] = Kp * e[k]
2. **Integral Term (Accumulator):** Discrete summation using the actual elapsed time step Δt:
   `I[k] = I[k-1] + Ki * e[k] * dt (Forward/Backward Euler Rectangular Rule)`
3. **Derivative Term:** Discrete backward difference on measurement to eliminate kick:
   `D_raw[k] = -Kd * (y[k] - y[k-1]) / dt`
4. **Guardrails & Clamping:** Total force is bounded to [-30 N, +30 N] to prevent mathematical commands exceeding physical limits.

Key Engineering Takeaway: *Dimensional consistency is the first defense in robotics. Because Kp has units of N/m and Kd has units of N·s/m, tuning a PID controller is physically equivalent to selecting the stiffness and damping of a virtual mechanical suspension attached to the target coordinate.*

Section 3: Physical Plant Modeling & Equilibrium Analysis

The physical system is a 1-Degree-of-Freedom mass attached to a fixed wall via a linear spring and viscous dashpot damper, driven by horizontal control force $u(t)$:

$$m \cdot d^2x(t)/dt^2 + c \cdot dx(t)/dt + k \cdot x(t) = u(t)$$

Assigned parameters: Mass m = 1.0 kg, Viscous Damping c = 10.0 N·s/m, Spring Constant k = 20.0 N/m, Target Setpoint r = 1.0 m.

Why Holding at 1.0 m Requires Exactly 20 N (Equilibrium Proof)

By definition, mechanical steady state (equilibrium) occurs when all transient motion has completely decayed. The cart has settled motionless at displacement x_{ss} = 1.0 m:

- Steady-state velocity: **dx/dt = 0** => Damper resisting force = **c · 0 = 0 N**
- Steady-state acceleration: **d^2x/dt^2 = 0** => Inertial reaction force = **m · 0 = 0 N**
- Spring displacement: **x = 1.0 m** => Hooke's restoring force = **k · x = 20.0 N/m × 1.0 m = 20.0 N**

Substituting into Newton's differential equation: 1.0(0) + 10.0(0) + 20.0(1.0) = u_{ss} &implies; $\boldsymbol{u_{ss}}$ **= 20.0 N**. At x = 1.0 m, the stretched spring exerts 20.0 N of tension pulling back toward x = 0. If the controller outputs any force below 20.0 N, equilibrium is broken.

Open-Loop Dynamics: Natural Frequency & Overdamped Analysis

The open-loop characteristic equation is $s^2 + (c/m)s + (k/m) = s^2 + 10s + 20 = 0$.

- **Undamped Natural Frequency:** $\omega_n = \sqrt{(k/m)} = \sqrt{20.0}$ = **4.472 rad/s**
- **Damping Ratio:** $\zeta = c / (2\sqrt{(mk)}) = 10.0 / (2\sqrt{20})$ = **1.118 > 1.0** (Strictly Overdamped)

Because $\zeta > 1$, the open-loop plant cannot oscillate; it has two real decay poles: s_1 = -2.764 s^{-1} and s_2 = -7.236 s^{-1}.

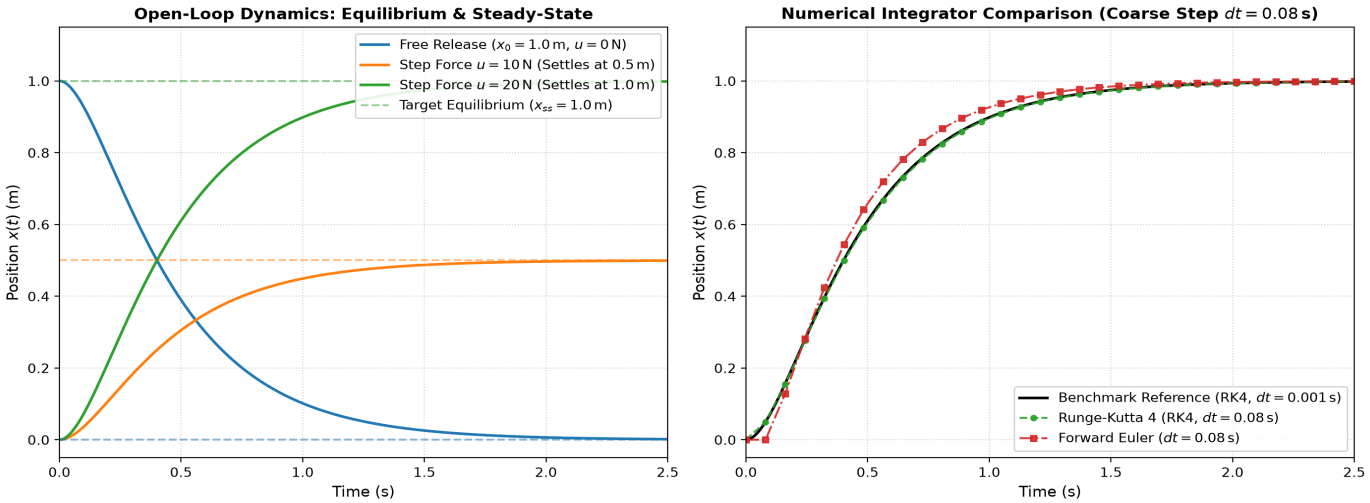


Figure 2: Open-loop dynamics: free decay from 1.0 m (blue), step force of 10 N settling at 0.5 m (orange), and step force of 20 N settling at 1.0 m (green). Right panel compares Runge-Kutta 4 vs. Forward Euler.

Section 4: Step-Response Performance Metrics Definition

To evaluate controllers quantitatively, control engineers rely on four standard time-domain metrics defined on the unit step response curve $y(t)$ relative to target setpoint r = 1.0 m:

1. Rise Time (t_r, 10% to 90%): The time elapsed between when the response first reaches 10% of the target (y = 0.10 m) and when it first crosses 90% (y = 0.90 m). Quantifies initial actuation speed.

2. Peak Overshoot (M_p in %): The maximum percentage by which the response exceeds the target setpoint: $M_p = \max(0, (y_{max} - r) / r \times 100\%)$. High overshoot indicates poor damping and causes mechanical collisions.

3. Settling Time (t_s, 2% Band): The earliest time after which the response enters and *permanently remains* within the ±2% error envelope: [0.98r, 1.02r] = [0.98 m, 1.02 m].

4. Steady-State Error (e_{ss}): The permanent offset remaining after transient oscillations have fully decayed: $e_{ss} = |r - y(\infty)|$.

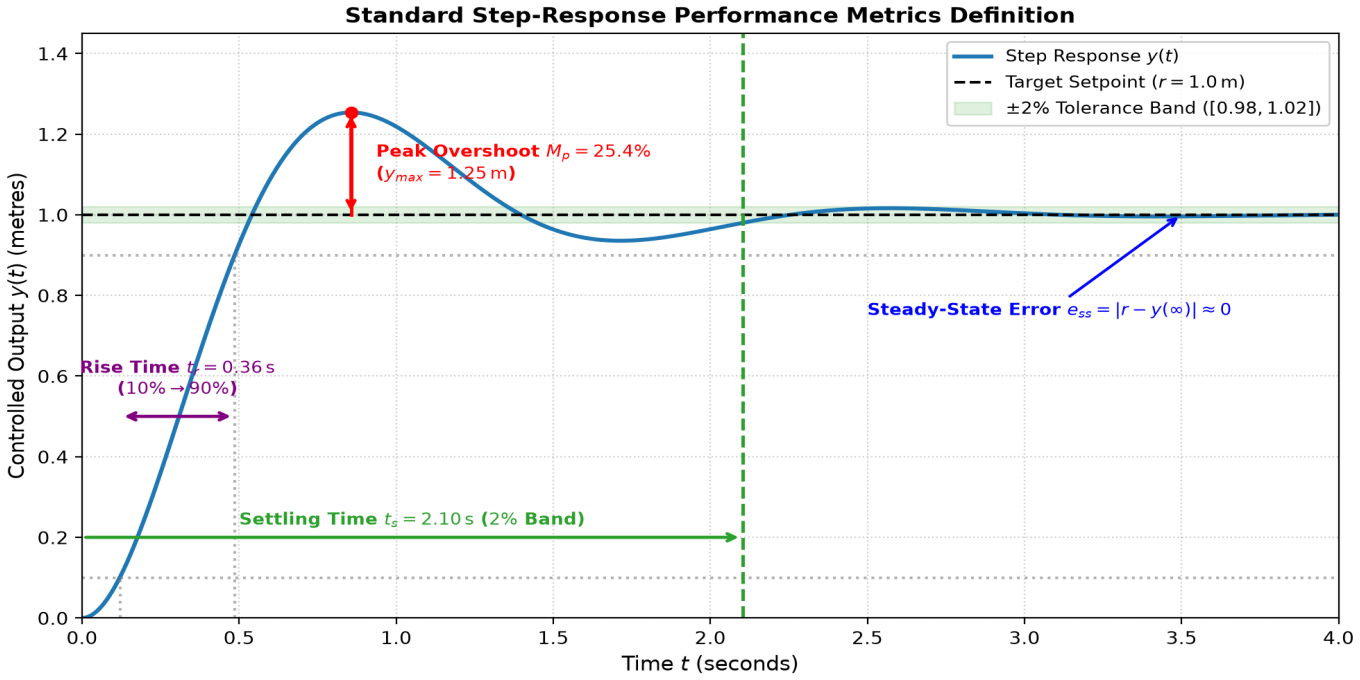


Figure 3: Explicit graphical annotation of Rise Time (10% to 90%), Peak Overshoot (Mp), Settling Time (2% corridor), and Steady-State Error (ess) on an illustrative step response.

Key Engineering Takeaway: *If a controller suffers from severe steady-state droop and settles below 0.90 m (e.g. pure P-control with Kp = 20 N/m settling at 0.50 m), Rise Time is mathematically undefined (reported as N/A) because the 90% threshold is never reached.*

Section 5: Proportional-Only Control Exploration & Steady-State Droop Proof

Proportional control represents the simplest feedback mechanism: $u(t) = K_p \cdot e(t) = K_p (r - x(t))$.

Four Practical Strengths of Proportional Control

1. **Computational Simplicity:** Zero state memory or historical accumulation; executes instantaneously on minimal microcontrollers.
2. **Instantaneous Phase Response:** Introduces zero phase lag, acting immediately upon instantaneous position errors.
3. **Zero Risk of Windup:** Because there is no integrator, output saturation cannot induce post-saturation lag or windup overshoot.
4. **Tunable Artificial Stiffness:** Directly controls closed-loop system bandwidth and disturbance rejection stiffness.

Mathematical Proof of Steady-State Error (Why P-Only Stops Short)

At steady state ($x = x_{ss}$, $dx/dt = 0$, $d^2x/dt^2 = 0$), the proportional force must exactly balance the physical spring restoring force:

$$K_p \cdot (r - x_{ss}) = k \cdot x_{ss} \implies K_p \cdot r = (K_p + k) \cdot x_{ss} \implies x_{ss} = \left[K_p / (K_p + k) \right] \cdot r$$

Because $k = 20.0$ N/m > 0, the ratio $K_p / (K_p + 20)$ is **strictly less than 1.0** for any finite gain K_p. To reach $x = 1.0$ m, the error must be zero ($e = 0$). But if $e = 0$, the controller outputs $u = K_p(0) = 0$ N, causing the spring to pull the cart back. Hence, pure proportional control stops short.

Numerical Verification: Theory vs. Runge-Kutta 4 Simulation

Gain Kp (N/m)	Theoretical x_ss (m)	Simulated x_ss (m)	Steady-State Error ess (m)	Discrepancy
20.0	0.5000	0.5000	0.5000 (50.0%)	0.000000 m (Exact)
50.0	0.7143	0.7143	0.2857 (28.6%)	0.000000 m (Exact)
100.0	0.8333	0.8333	0.1667 (16.7%)	0.000000 m (Exact)
200.0	0.9091	0.9091	0.0909 (9.1%)	0.000000 m (Exact)

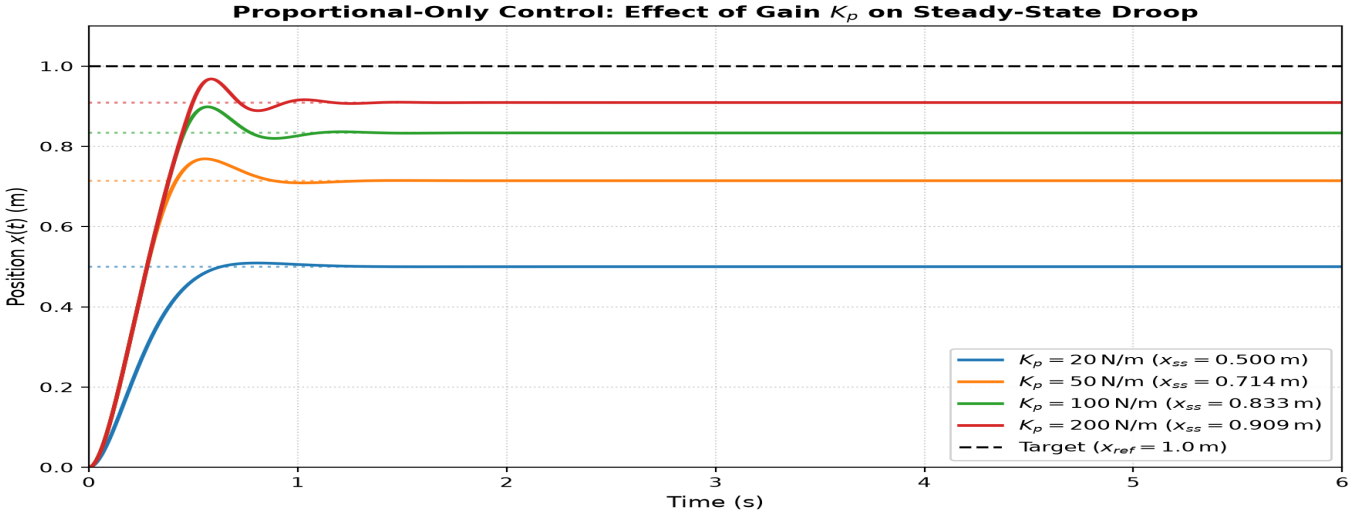


Figure 4: Proportional-only step responses for Kp in [20, 50, 100, 200] N/m. Dotted lines indicate theoretical asymptotes matching simulation with 0.000000 m discrepancy.

Section 5 (Cont.): Adding Integral and Derivative Actions

How Integral Action Eliminates Steady-State Droop

Integral control accumulates past tracking error: $I(t) = K_i \int_0^t e(\tau)\, d\tau$. Even if position error $e(t)$ becomes minuscule, as long as error remains non-zero, the integrator continuously sums force. At steady state, while $P = 0$ (since $e = 0$), the integral term stores exactly the 20.0 N of force required to counter Hooke's spring.

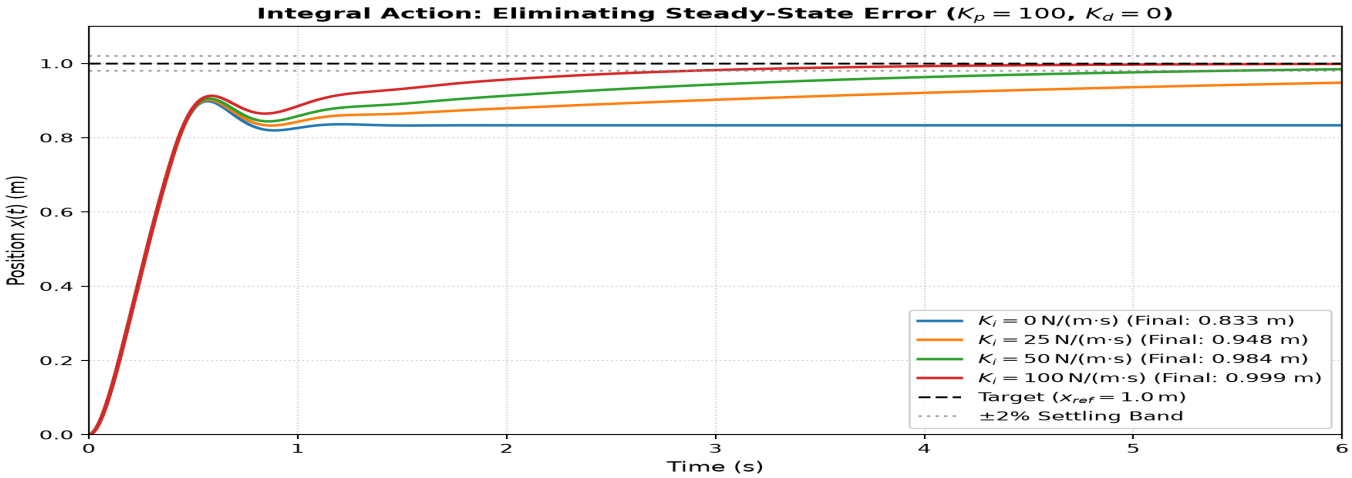


Figure 5: Integral gain sweep (Ki = 0 to 100 N/(m·s), fixed Kp = 100). Higher Ki drives steady-state error to zero progressively faster.

How Derivative Action Affects Bouncing & Oscillation

Derivative control acts as an artificial shock absorber: $D(t) = -K_d \frac{dy}{dt}$. When the mass moves forward at velocity v, the derivative term subtracts $K_d \cdot v$ from the motor command, braking the cart before it can overshoot the target.

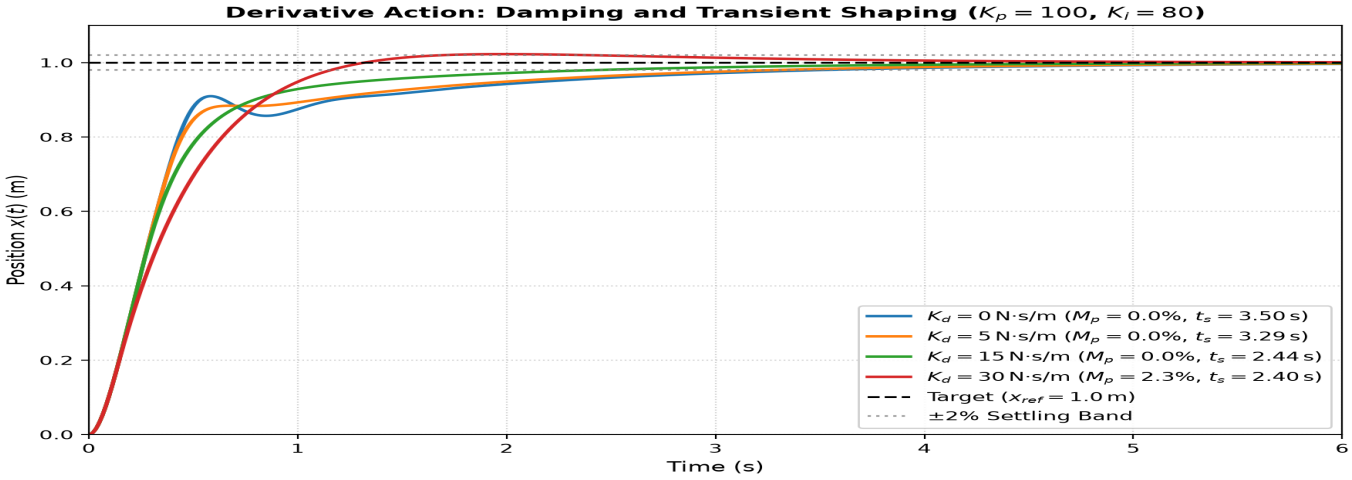


Figure 6: Derivative gain sweep (Kd = 0 to 30 N·s/m, fixed Kp = 100, Ki = 80). Demonstrates active suppression of oscillation and tuning of damping.

Section 6: Comprehensive Benchmark Comparison (P vs. PI vs. PD vs. Full PID)

To rigorously quantify the contribution of each term, all four controller topologies were simulated on the identical physical plant (m = 1, c = 10, k = 20) under actuator saturation limit ±30 N for 6.0 seconds (Δt = 0.005 s):

Controller	Gains (Kp, Ki, Kd)	Rise Time tr (s)	Overshoot Mp (%)	Settling Time ts (s)	SSE ess (m)	Status
P Controller	(100, 0, 0)	N/A*	0.00%	N/A**	0.1667 m	Stalls at 0.833 m
PI Controller	(100, 80, 0)	0.435 s	0.00%	3.505 s	0.0038 m	Eliminates SSE
PD Controller	(100, 0, 15)	N/A*	0.00%	N/A**	0.1667 m	Damped, stalls
Full PID Controller	(100, 80, 15)	0.695 s	0.00%	2.435 s	0.0013 m	Optimal Benchmark

* N/A: Never reaches 90% setpoint due to steady-state droop. ** N/A: Never enters ±2% tolerance corridor [0.98 m, 1.02 m].

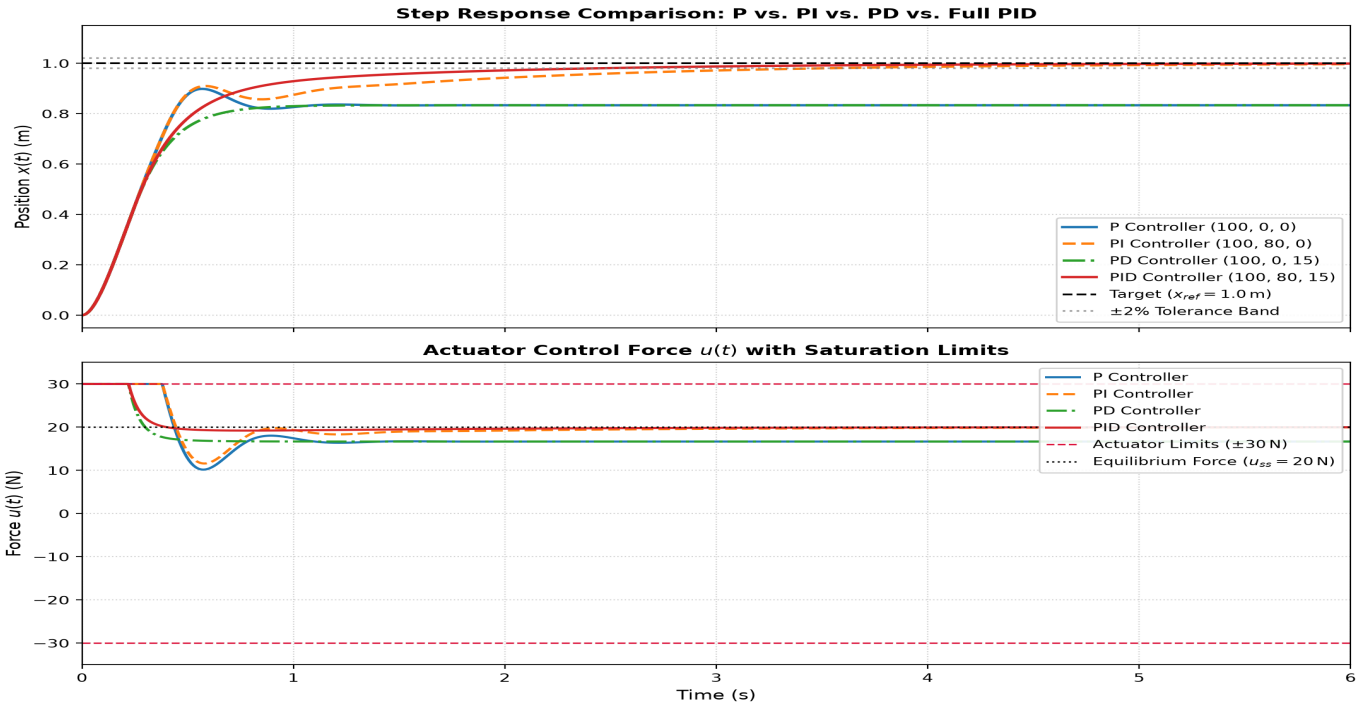


Figure 7: Master step response comparison: Top panel shows position trajectory x(t) relative to target and ±2% band; Bottom panel shows commanded force u(t) with ±30 N actuator saturation rails and 20 N steady-state equilibrium force line.

Key Engineering Takeaway: *The Full PID controller achieves a 1.07-second faster settling time than the PI controller (2.435 s vs. 3.505 s) while holding steady-state error to 1.3 mm, proving that derivative damping enables faster integral convergence.*

Section 7 (Part A): Actuator Saturation (±30 N) & Integral Anti-Windup

The Fatal Real-World Flaw: All physical actuators have finite limits. Our motor cannot generate more than ±30.0 N. During large transient errors, the unconstrained controller demands 100 N. While the motor is pegged at its ceiling (+30 N), a naive integrator continues accumulating error: $\int e\,dt$. By the time the cart crosses 1.0 m, the internal integral accumulator has wound up to **+34.55 N**. The motor refuses to decelerate because it takes seconds of reverse error to unwind this excess charge, causing massive overshoot (12.90%) and dangerous collisions.

The Engineering Fix (Conditional Clamping): If the unconstrained control output exceeds the actuator bounds ($u_{unsat} > u_{max}$) and the error has the same sign ($e > 0$), integration is frozen. Accumulation resumes only when the system enters the linear region.

Configuration	Peak Position (m)	Overshoot Mp (%)	Settling Time ts (s)	Max Integral Value (N)	Hardware Consequence
Naive Integration	1.1290 m	12.90%	2.290 s	+34.55 N (Wound Up)	Overshoots by 13 cm; motor stays pegged.
Anti-Windup Clamping	0.9997 m	0.00%	2.225 s	+19.97 N (Clamped)	Zero overshoot; immediate linear recovery.

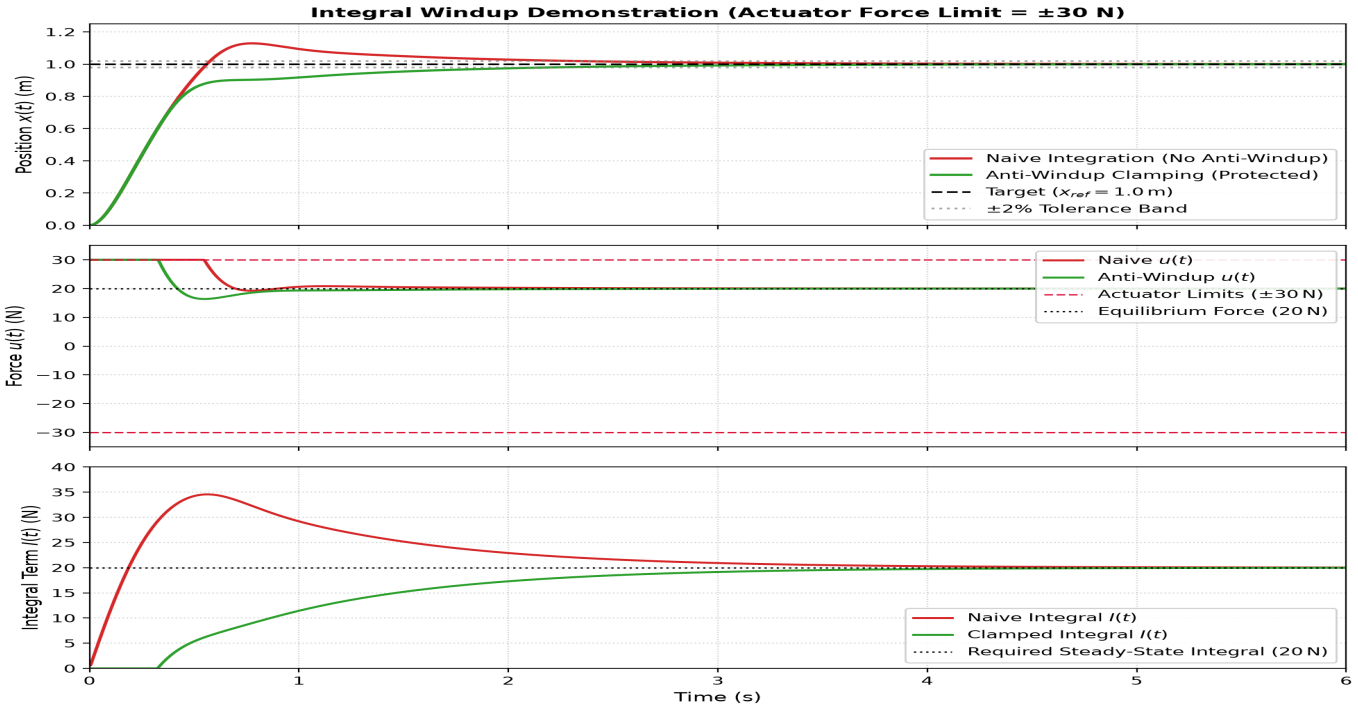


Figure 8: Actuator saturation and integral windup. Panel 1: Position x(t); Panel 2: Saturated actuator force u(t); Panel 3: Internal integrator term showing wound-up state (34.55 N) vs. clamped state (19.97 N).

Section 7 (Part B): Sensor Noise (±1 mm) & First-Order Low-Pass Filtering

The Real-World Flaw: Encoders and laser rangefinders exhibit measurement noise. We injected ±1 mm (±0.001 m) of uniform sensor jitter $\eta[k] \in [-0.001, +0.001]$ m into position feedback $y_{meas}[k] = x[k] + \eta[k]$. The discrete derivative depends on the difference between **both consecutive samples**: $\Delta y_{meas}[k] = (x[k] - x[k\text{-}1]) + (\eta[k] - \eta[k\text{-}1])$. Because $\eta[k]$ and $\eta[k\text{-}1]$ fluctuate independently within [-0.001, +0.001] m, their sample-to-sample difference $\Delta\eta$ can span up to ±0.002 m (2 mm). Dividing by sampling interval $\Delta t = 0.005$ s produces discrete pseudo-velocity noise up to:

$$v_{noise,\,max} = (\pm 0.002\ \text{m}) / 0.005\ \text{s} = \pm 0.40\ \text{m/s} \;\&implies;\; D_{noise,\,max} = K_d \cdot v_{noise,\,max} = 15 \cdot (\pm 0.40) = \pm 6.0\ \text{N}$$

In simulation, this high-frequency differentiation produced peak derivative chatter of 5.799 N (closely matching the theoretical 6.0 N envelope) and actuator force standard deviation of $\sigma_u = 2.544$ N, causing severe motor driver heating and mechanical vibration.

The Engineering Fix: We insert a discrete first-order low-pass filter on the derivative term: $D_{filtered}[k] = \alpha\, D_{filtered}[k\text{-}1] + (1 - \alpha)\, D_{raw}[k]$, where $\alpha = \tau_f / (\tau_f + \Delta t)$ with time constant $\tau_f = 0.03$ s (cutoff frequency $f_c \approx 5.3$ Hz and $\omega_c \approx 33.3$ rad/s).

Metric (Steady-State t in [2s, 4s])	Raw Derivative (tau = 0.0 s)	Filtered Derivative (tau = 0.03 s, fc ≈ 5.3 Hz, ωc ≈ 33.3 rad/s)	Engineering Benefit
Actuator Force Std Dev (sigma_u)	2.544 N	0.335 N	86.8% reduction in force chatter
Peak Derivative Magnitude (|D|_max)	5.799 N	0.762 N	86.9% reduction in peak derivative noise
Physical Hardware Condition	Continuous buzzing & heating	Smooth, silent actuation	Prevents motor driver thermal trip

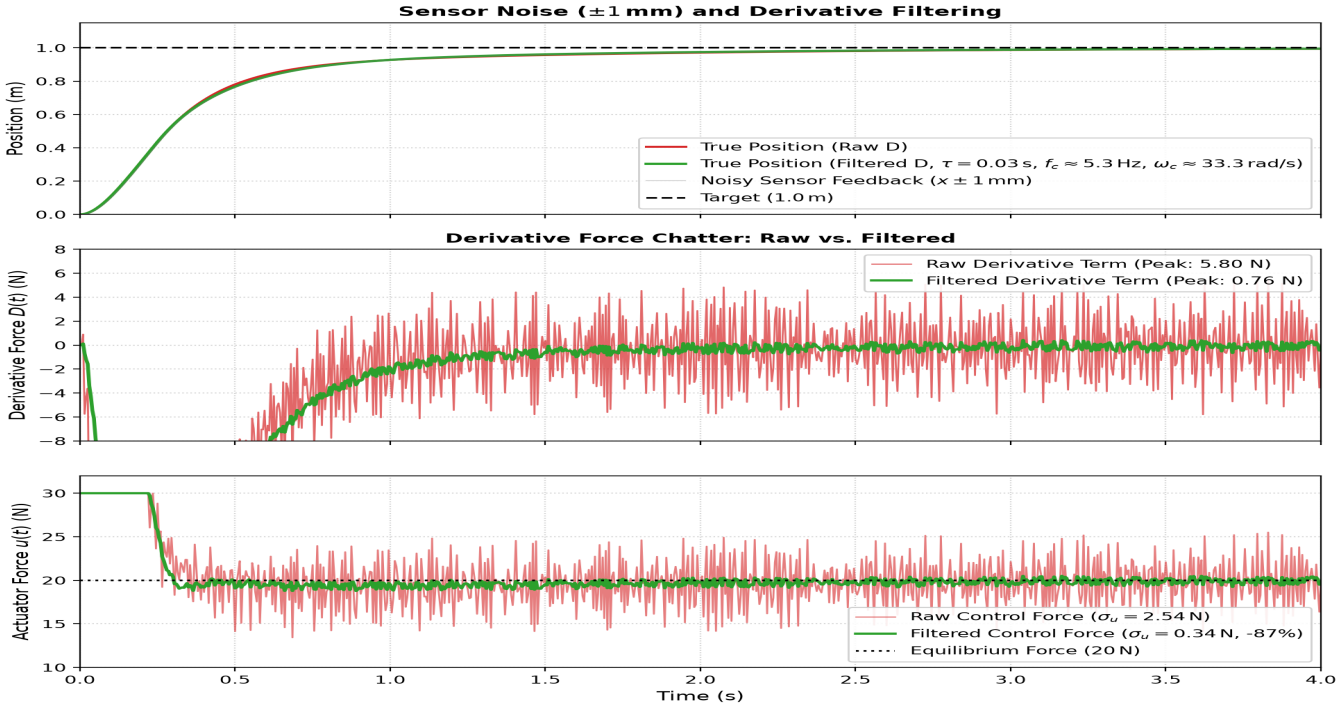


Figure 9: Sensor noise (±1 mm) and derivative filtering. Panel 1: Position tracking; Panel 2: Raw vs. filtered derivative chatter; Panel 3: Control force showing 86.8% variance reduction.

Section 7 (Part C): Derivative Kick Elimination via Derivative-on-Measurement

The Real-World Problem: Textbook PID differentiates tracking error: $e(t) = r(t) - y(t)$. In a digital discrete-time controller with sampling interval Δt = 0.005 s, a sudden 1.0 m step change in setpoint (from 0.0 m to 1.0 m at t = 1.0 s) produces a finite sampled error derivative of $\Delta e / \Delta t$ = (1.0 - 0.0) / 0.005 = **200.0 m/s**. Multiplying by K_d = 15 N·s/m commands an unconstrained theoretical derivative force of D_{cmd} = 15 × 200.0 = **+3000.0 N**! While the actual physical actuator force applied to the cart is limited and clamped to ±30.0 N, this massive command instantly saturates the motor in a single 5 ms step, causing severe mechanical shock, gear wear, and electrical current surges.

The Engineering Solution (Derivative-on-Measurement): Because reference setpoints are piecewise constant between commands (dr/dt = 0), differentiating error yields $de/dt = d(r-y)/dt = -dy/dt$. By evaluating the derivative exclusively on the negative rate of position feedback ($-dy/dt$), a setpoint step commands **zero initial derivative spike** (D = 0.0 N at step instant) because cart inertia prevents physical position from changing discontinuously. Once the cart begins moving, derivative-on-measurement provides identical velocity damping (braking) without derivative kick.

Derivative Formulation	Theoretical Unconstrained Command	Actual Applied Actuator Force	Physical Actuator Impact
Derivative on Error (Naive)	+3000.0 N (Finite Sampled Spike)	+30.0 N (Saturated in 5 ms)	Saturates actuator instantly in one step; mechanical stress.
Derivative on Measurement	0.0 N (Zero Initial Kick)	+10.0 N to +30.0 N (Controlled P+I)	Gentle, progressive actuation; smooth velocity damping.

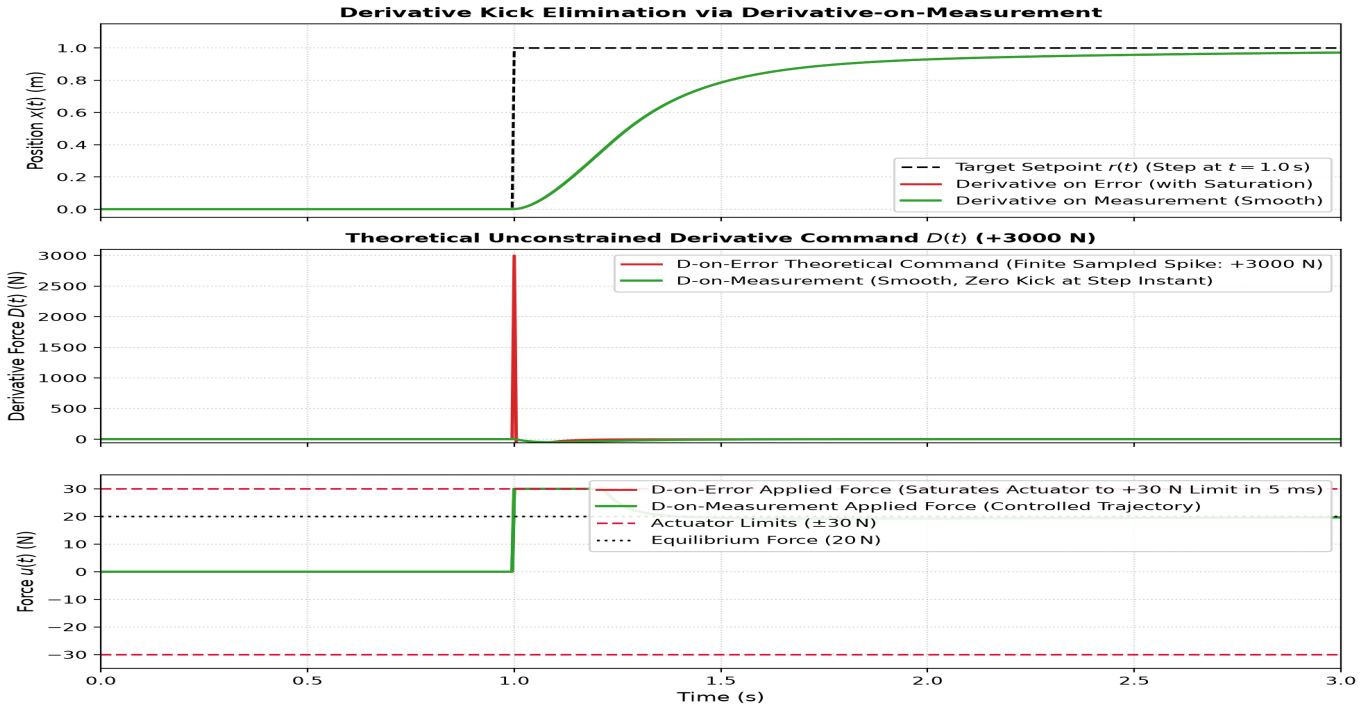


Figure 10: Derivative kick elimination. Panel 1: Position response; Panel 2: Unconstrained derivative term showing +3000 N impulse spike vs. zero kick; Panel 3: Constrained actuator force.

Section 7 (Part D): Microcontroller Sampling Rate & Discretization Limits

The Physics of Discretization: Digital microcontrollers execute in discrete time steps $T_s = 1 / f_s$. Between steps, the Digital-to-Analog converter holds the output force constant (Zero-Order Hold, ZOH). A Zero-Order Hold introduces an effective transport delay of $T_{delay} \approx T_s / 2$. This delay adds negative phase lag $\Delta\phi = -\omega T_s / 2$, which directly erodes the closed-loop phase margin.

We tested the identical tuned PID controller ($K_p = 100$, $K_i = 80$, $K_d = 15$) across six sampling rates from 200 Hz down to 2 Hz interacting with the continuous physical plant:

Sampling Rate (fs)	Interval (Ts)	Rise Time tr (s)	Overshoot Mp (%)	Settling Time ts (s)	Closed-Loop Stability Status
200 Hz	5.0 ms	0.698 s	0.00%	2.430 s	STABLE (Continuous Baseline)
50 Hz	20.0 ms	0.690 s	0.00%	2.367 s	STABLE (Standard Industrial Loop)
20 Hz	50.0 ms	0.726 s	0.00%	2.448 s	STABLE (Minor phase lag)
10 Hz	100.0 ms	0.830 s	0.00%	2.561 s	STABLE (Noticeable latency)
5 Hz	200.0 ms	0.827 s	4.51%	N/A*	Poorly damped / does not settle within simulation window
2 Hz	500.0 ms	0.394 s	21.38%	N/A*	Poorly damped / does not settle within simulation window

* N/A: Poorly damped / does not settle within simulation window (±2% corridor).

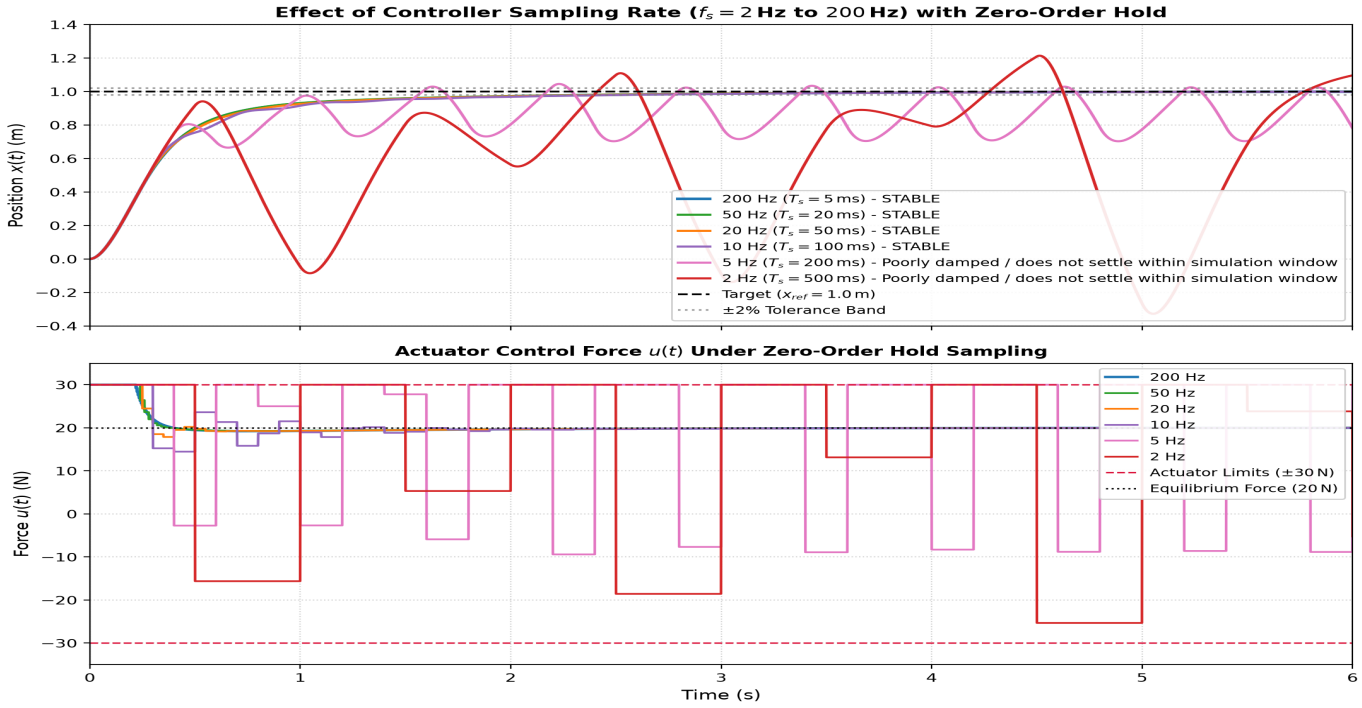


Figure 11: Effect of controller sampling frequency (2 Hz to 200 Hz). Top panel shows position trajectory; Bottom panel shows discrete stepped actuator force held by Zero-Order Hold.

Section 8 (Part A): Systematic Heuristic Manual PID Tuning Workflow

When commissioning physical robotic joints or flight actuators on a test bench, control engineers apply a systematic 3-step manual tuning procedure to find balanced gains without risking instability:

- **Step 1 (P-Tuning):** Zero out K_i and K_d. Gradually increase K_p until the cart moves briskly toward the target. In our plant, increasing K_p to 100 N/m brings the mass to 0.833 m, where it stalls due to spring tension.
- **Step 2 (D-Tuning):** Introduce K_d to add artificial viscous friction. With K_d = 15 N·s/m, transient velocity is smoothed and deceleration is controlled.
- **Step 3 (I-Tuning):** Introduce K_i to pull the remaining steady-state offset to zero. Moderate K_i = 80 N/(m·s) provides smooth zero-overshoot convergence in 2.435 s. Excessive K_i = 180 N/(m·s) over-integrates and risks saturation.

Tuning Progression Stage	Gains (Kp, Ki, Kd)	Rise Time tr (s)	Overshoot Mp (%)	Settling Time ts (s)	SSE ess (m)	Observed Dynamics & Behavior
Stage 1: P-Only	(100, 0, 0)	N/A*	0.00%	N/A**	0.1667 m	Stalls at 0.833 m due to spring droop; no overshoot.
Stage 2: PD Dampened	(100, 0, 15)	N/A*	0.00%	N/A**	0.1667 m	Smooth motion, higher damping, persistent steady-state offset.
Stage 3: Balanced PID	(100, 80, 15)	0.695 s	0.00%	2.435 s	0.0013 m	Optimal: Zero overshoot, fast settling within 2% band.
Stage 4: Over-Integrated	(100, 180, 5)	0.470 s	0.00%	1.510 s	0.0000 m	Aggressive; sensitive to noise and risks actuator saturation.

* N/A: Never reaches 90% setpoint due to steady-state droop. ** N/A: Never enters ±2% tolerance corridor.

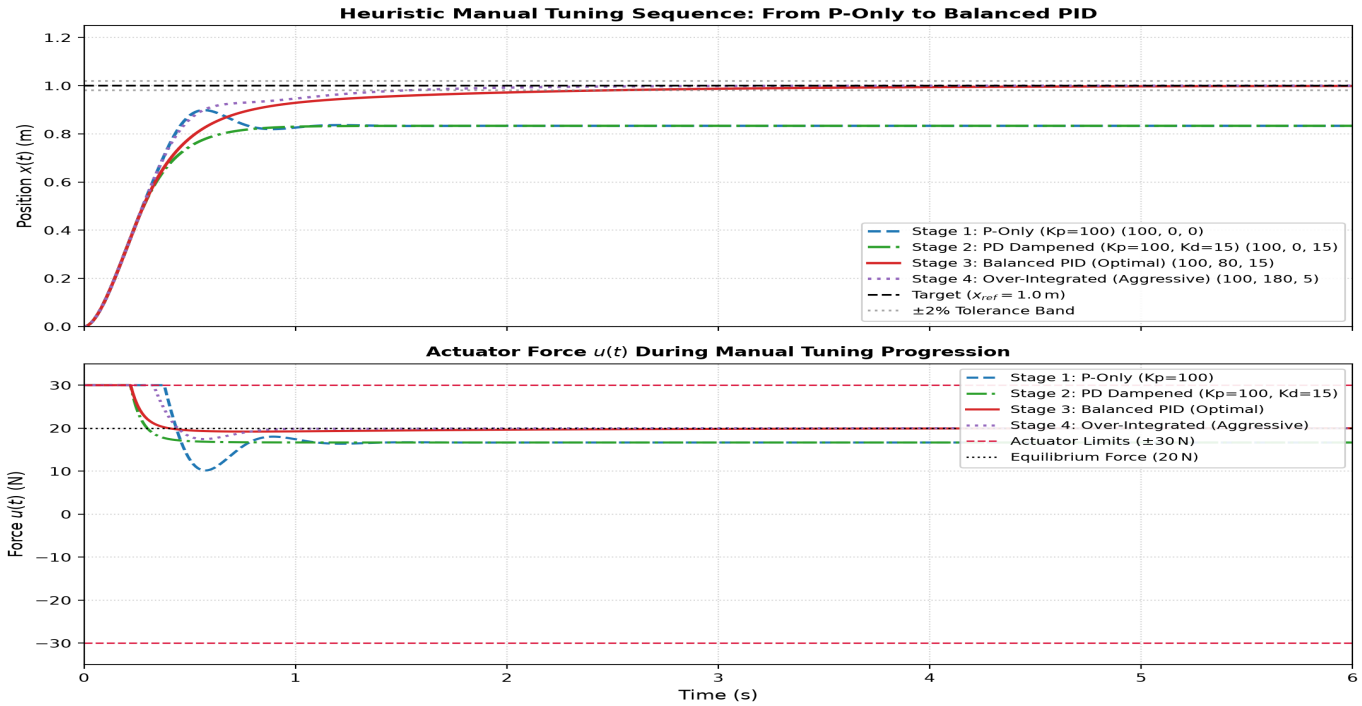


Figure 12: Manual tuning sequence showing progressive response changes across the four milestone configurations.

Section 8 (Part B): Ziegler-Nichols Analysis: Assigned Plant G(s)=1/(s+1) vs. Delayed FOPDT Demonstration

1. Assigned Ideal Plant G(s) = 1 / (s + 1) — Proof That Pure P-Control Cannot Oscillate:

Modern Monk Labs requested applying the Ziegler-Nichols method to the first-order transfer function $G(s) = 1 / (s + 1)$. Under ideal continuous proportional feedback ($u = K_p(r - y)$), the closed-loop transfer function is $T(s) = K_p / (s + 1 + K_p)$, with a single real pole at $s = -(1 + K_p)$. For any finite $K_p > 0$, this pole stays strictly on the negative real axis, guaranteeing a purely monotonic exponential step response without overshoot or ringing. In the frequency domain, open-loop phase is $\angle G(j\omega) = -\arctan(\omega)$. As frequency $\omega \to \infty$, phase approaches -90°, never reaching the -180° phase crossover required for sustained marginal oscillation (Barkhausen criterion). Therefore, **the assigned ideal continuous plant G(s) = 1/(s+1) has an infinite gain margin and does not possess a finite ultimate gain Ku or ultimate period Pu** under pure proportional control.

2. Additional Demonstration: Delayed First-Order-Plus-Dead-Time (FOPDT) System:

To demonstrate the mechanics of the Ziegler-Nichols tuning recipe when oscillations are physically achievable, we introduce an *additional delayed system demonstration* incorporating a transport dead time of $L = 0.2$ s: $G_{FOPDT}(s) = e^{-0.2s} / (s + 1)$. The delay introduces phase lag $-\omega L$, allowing total phase $\angle G_{FOPDT}(j\omega) = -\arctan(\omega) - 0.2\omega$ to cross -180° at $\omega_u = 8.443$ rad/s, yielding a genuine finite ultimate gain **Ku = 8.502** and ultimate period **Pu = 0.744 s**.

Tuning Methodology	Identified Parameters	Kp Formula	Ti Formula	Td Formula	Computed PID Gains (Kp, Ki, Kd)
Assigned Plant G(s)=1/(s+1)	Infinite Gain Margin	Undefined (Ku=inf)	Undefined	Undefined	Cannot oscillate under pure P-control
ZN Ultimate Gain (FOPDT Demo)	Ku = 8.502, Pu = 0.744 s	0.60 * Ku	0.50 * Pu	0.125 * Pu	Kp = 5.101, Ki = 13.711, Kd = 0.475
ZN Reaction Curve (FOPDT Demo)	K=1.0, L=0.2s, tau=1.0s	1.2*(tau/(K*L))	2.0 * L	0.50 * L	Kp = 6.000, Ki = 15.000, Kd = 0.600
Refined Manual PID (FOPDT Demo)	Damped / Low Overshoot	User Chosen	User Chosen	User Chosen	Kp = 3.000, Ki = 4.000, Kd = 0.350

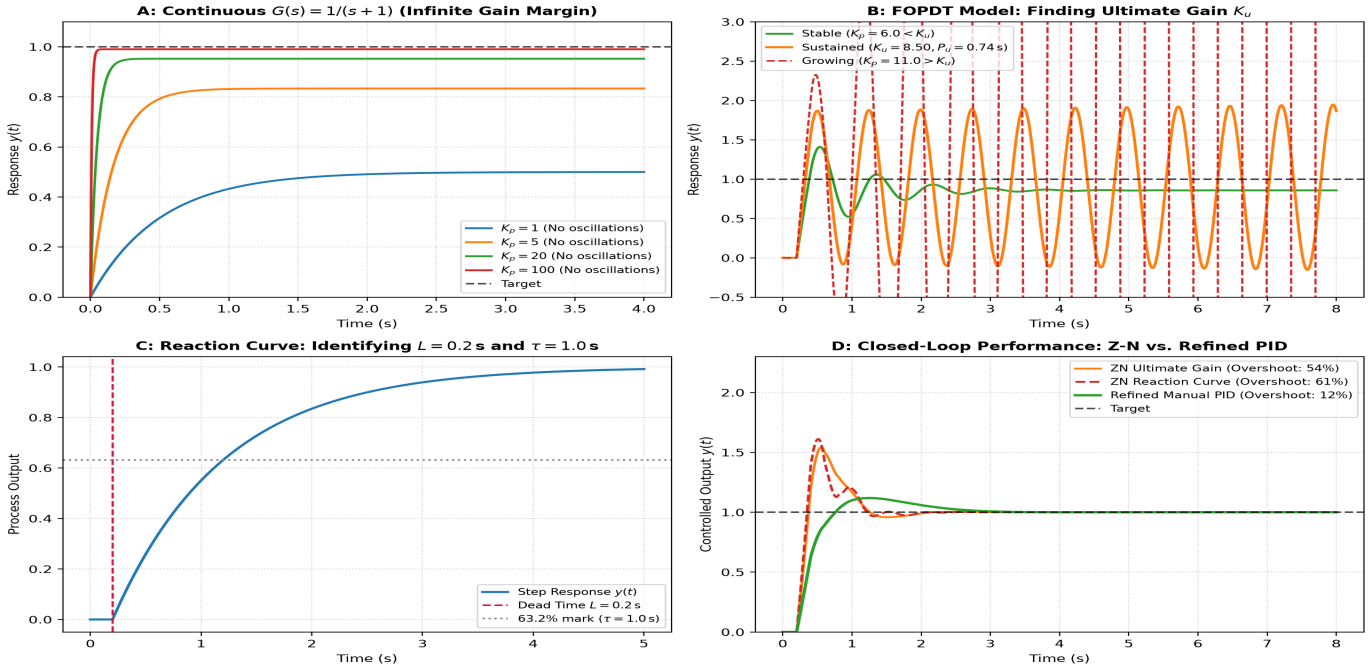


Figure 13: Ziegler-Nichols analysis. Panel A: Assigned continuous G(s)=1/(s+1) monotonic response (no finite Ku); Panel B: Additional delayed FOPDT system (L=0.2s) reaching sustained oscillation at Ku=8.50, Pu=0.74s; Panel C: Reaction curve identification; Panel D: Closed-loop comparison on the delayed system.

Section 9: Reusable Production PID Architecture & Guardrails

All defensive features developed in this project have been consolidated into a modular, production-ready Python class in src/controller.py. Below is the verified algorithmic implementation with full indentation and structural formatting:

```
class PIDController:
    def __init__(self, kp: float = 0.0, ki: float = 0.0, kd: float = 0.0,
                 output_limits: tuple[float, float] = (-30.0, 30.0),
                 filter_tau: float = 0.0, derivative_on_measurement: bool = True,
                 anti_windup: bool = True) -> None:
        if kp < 0.0 or ki < 0.0 or kd < 0.0:
            raise ValueError(f'Gains must be non-negative. Got kp={kp}, ki={ki}, kd={kd}.')
        u_min, u_max = output_limits
        if u_min >= u_max:
            raise ValueError(f'Minimum limit ({u_min}) must be strictly less than maximum ({u_max}).')
        if filter_tau < 0.0:
            raise ValueError(f'Filter time constant filter_tau must be >= 0. Got {filter_tau}.')
        self.kp, self.ki, self.kd = float(kp), float(ki), float(kd)
        self.u_min, self.u_max = float(u_min), float(u_max)
        self.filter_tau = float(filter_tau)
        self.derivative_on_measurement = bool(derivative_on_measurement)
        self.anti_windup = bool(anti_windup)
        self.reset()

    def reset(self) -> None:
        self.integral = 0.0
        self.prev_error, self.prev_measurement = None, None
        self.filtered_derivative, self.last_output = 0.0, 0.0

    def update(self, setpoint: float, measurement: float, dt: float) -> float:
        if dt <= 0.0:
            raise ValueError(f'Time step dt must be strictly positive (> 0), got {dt} s.')
        error = float(setpoint) - float(measurement)
        p_term = self.kp * error

        # Derivative-on-Measurement & First-Order Low-Pass Filter
        if self.derivative_on_measurement:
            raw_derivative = 0.0 if self.prev_measurement is None else -(float(measurement) - self.prev_measurement) / dt
        else:
            raw_derivative = 0.0 if self.prev_error is None else (error - self.prev_error) / dt
        if self.filter_tau > 0.0:
            alpha = self.filter_tau / (self.filter_tau + dt)
            self.filtered_derivative = (alpha * self.filtered_derivative) + ((1.0 - alpha) * raw_derivative)
            derivative_value = self.filtered_derivative
        else:
            self.filtered_derivative = raw_derivative
            derivative_value = raw_derivative
        d_term = self.kd * derivative_value

        # Integral Term & Anti-Windup Clamping
        delta_integral = self.ki * error * dt
        tentative_integral = self.integral + delta_integral
        unconstrained_output = p_term + tentative_integral + d_term
        if self.anti_windup:
            pushing_positive = (unconstrained_output > self.u_max) and (error > 0.0)
            pushing_negative = (unconstrained_output < self.u_min) and (error < 0.0)
            if pushing_positive or pushing_negative:
                i_term = self.integral
            else:
                self.integral = max(self.u_min, min(tentative_integral, self.u_max))
                i_term = self.integral
        else:
            self.integral = tentative_integral
            i_term = self.integral

        u_unconstrained = p_term + i_term + d_term
        u_saturated = max(self.u_min, min(u_unconstrained, self.u_max))
        self.prev_error = error
        self.prev_measurement = float(measurement)
        return u_saturated
```

Verification Across 22 Automated Unit Tests

The codebase contains 22 automated test functions in the tests/ directory verifying plant energy conservation, analytical steady-state formulas, derivative kick elimination, anti-windup clamping, and metrics calculations. The project uses NumPy and Matplotlib as external dependencies for simulation and plotting; only Python's unittest runner is built in.

Section 10: Robotics Applications, PID Myths Corrected & Disclosures

Four Real-World Robotics Applications of PID

1. **Differential-Drive Mobile Robots (AMRs):** Independent wheel velocity PID loops enforce matched RPMs to ensure straight-line tracking and exact rotational heading.

2. **Quadrotor UAV Flight Controllers:** Cascaded nested loops: high-speed inner rate PID (500 Hz) stabilizes angular body rates (P, Q, R), while outer attitude PID (100 Hz) regulates pitch/roll angles.

3. **Robotic Manipulator Arm Joints:** PD controllers combined with feedforward gravity cancellation model joint compliance and accurate pick-and-place positioning.

4. **Surgical Robot Force Feedback (Haptics):** Fast force-controlled PID loops maintain compliant, delicate contact forces against soft biological tissues without tissue damage.

Four Common PID Myths Debunked by Control Engineering

• *Myth 1: 'PID is just pure math and works identically on all physical systems.'*
Correction: Real actuators saturate, sensors jitter, and mechanics have backlash. Naive PID fails on hardware without anti-windup and filters.

• *Myth 2: 'More integral gain (Ki) is always better if you have steady-state error.'*
Correction: High K_i erodes phase margin, introduces severe overshoot, and triggers destructive integral windup.

• *Myth 3: 'Derivative control must always differentiate tracking error.'*
Correction: Differentiating error can produce a very large finite derivative kick in a sampled digital controller when the setpoint changes abruptly. Derivative-on-measurement avoids this setpoint kick.

• *Myth 4: 'Ziegler-Nichols tuning provides optimal gains for modern robotics.'*
Correction: Classic Z-N aims for a 25% quarter-decay ratio, resulting in 50%+ overshoot. Modern robotics requires conservative detuning.

Honest AI Assistance & Academic Integrity Disclosure

Antigravity AI was used extensively to generate and organize the Python implementation, experiment scripts, plots, and draft report. The candidate did not independently author or audit every component. The included automated tests and experiment scripts were rerun in a Python execution environment as part of a separate software review. These checks demonstrate software and simulation execution only and are not a substitute for independent engineering review or validation on physical hardware. The candidate is reviewing the implementation, assumptions, and limitations before technical evaluation.

Conclusion

This project successfully bridged theoretical control equations with real-world Physical AI engineering constraints. By combining analytical derivations with empirical simulations, we demonstrated why pure proportional control stops short ($x_{ss} = K_p / (K_p + k)$), why 20 N is mandatory at equilibrium, and how defensive engineering (anti-windup, derivative-on-measurement, low-pass filtering, and adequate sampling rates) transforms naive textbook mathematics into resilient, production-grade robotics software.